Polymorphism

① Derived from Greek meaning "having many forms".

Concept: Allows client code to treat objects (sub classes) as the same general type. At runtime, each specific object executes its own correct behavior.

Bad Practice (Without Polymorphism): Using manual type-checking (instanceof) and casting inside dynamic [if/else] loops.
It makes code verbose, error-prone, and hard to maintain when adding new classes.

Implementation Steps:

1. Define a common interface or abstract class with abstract methods [ChessPiece with getPossibleMoves()].

2. Implement/Override the method in each concrete sub class (Pawn, Rook)

3. Depend on the superclass in client code, invoking the common method directly.

Trade-offs:

Pros: Shorter, cleaner, and highly extensible client code (new subclasses can be added without changing client code).

Cons: Specific behavior is determined at runtime, shifting static predictability to runtime flexibility.

Java Specifics:

Overriding (@Override): Redefining an inherited method in a subclass

Overloading: Defining multiple methods with the same name but different parameters in the same class.

Abstraction:

Focusing only on the relevant attributes/behaviors of an object for a given system while ignoring unnecessary details.

Hierarchy: A general superclass (ChessPiece) is more abstract than a concrete subclass (Pawn) because it contains fewer specific details.

System Design Strategy:

Identify shared similarities across different systems/classes (movement on a grid, picking up items).

Create an abstract layer containing those common elements.

Delegate specific differences to concrete subclasses.

Key note: An abstract class/system cannot stand on its own whitout concrete implementations.

Process vs Artifact:

- As a process, abstraction is the act of bringing relevant details to the forefront while suppressing irrelevant ones within a given context.

- As a software artifact, an abstraction is a description (like a class deffinition) that exposes usable details while omitting unnecessary implementation logic.

Context Dependency: The properties included in a class depend entirely on the system's purpose. For example, a "Person" class in an airline reservation system requires different properties (pasport, seats) than in a soccer tournament system (player, position, stats), just as a Chicken class differs between a farm management system and a coop simulator

Java Constructs: Abstract classes cannot be instantiated, and abstract methods force derived classes to provide implementations. Interfaces define reusable public contracts that concrete classes implement using keywords like (implements) and (@Override).

Interfaces:

1. User Research and Complexity.

UX Research :: Emphasizes observing user behavior rather than just listening to their expressed attitudes, as stated altitudes may change during requirements gathering.

UI Complexity: Distinguishes between Essential Complexity (inherent to the problem) and Accidental Complexity (introduced by design/technology decisions). The core goal is to handle essential complexity while minimizing accidental complexity ("Keep the Simple Simple")

2. Raskin's Four Laws of Humane Interface:

1st law: A computer shall not harm your work or, through inaction, allow your work to come to harm.

2nd law: A computer shall not waste your time or require you to do more work than is strictly necessary.

3rd law: An interface is humane if it is responsive to human needs and considerate of human frailties.

4th law: The user should set the pace of interaction.

3.- Design Methodologies

Effective Aproaches:

- Human.-centered Design: Aligns interfaces with human physiological facts
- User-centered Design: Driven by an accurate understanding of users needs, desires, and limitations.
- Usage centered Design: Based on user intentions and actual usage patterns.
- Related Standards: Goal directed Design. Usability Engineering, and interaction Design.

Ineffective Approaches: Driving design based strictly on database structures, progamming language/development environments, or stecific platforms/frameworks.